

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені Тараса Шевченка
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
Кафедра програмних систем і технологій

Дисципліна
« Конструювання програмного забезпечення »

Лабораторна робота № 5

Виконав:	Берещук Назар Олександрович	Перевірів:	Шабатін П. Є.
Група	ІПЗ-34-10	Дата перевірки	
Форма навчання	денна	Оцінка	
Спеціальність	121		

2026

Варіант 1: Симулятор системи залежностей (Dependency Resolver)

Етап 1 (Docker):

Для ізоляції середовища виконання було створено файл Dockerfile, який описує процес збірки та запуску застосунку.

```
FROM maven:3.9-eclipse-temurin-25 AS build

WORKDIR /app

COPY pom.xml .
COPY src ./src
COPY config.json .

RUN mvn clean package -DskipTests

FROM eclipse-temurin:25-jre

WORKDIR /app

COPY --from=build /app/target/PackageManager-1.0-SNAPSHOT.jar app.jar
COPY --from=build /app/config.json config.json

ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Було використано підхід multi-stage build, що дозволяє розділити:

1. етап збірки (з використанням Maven та JDK),
2. етап виконання (з використанням лише JRE).

На першому етапі контейнер збирає проєкт за допомогою Maven. На другому етапі копіюється лише готовий jar файл та конфігурація.

Також було створено файл dockerignore, який виключає зайві файли з контексту збірки.

```
target
.git
.idea
*.iml
*.log
README.md
out
```

Етап 2 (CI):

Було налаштовано GitHub Actions workflow у директорії віддаленого репозиторію `.github/workflows/maven-ci.yml`.

```
1   name: Maven CI
2
3   on:
4     push:
5       branches:
6         - main
7     pull_request:
8       branches:
9         - main
10
11  jobs:
12    build-and-test:
13      runs-on: ubuntu-latest
14
15      steps:
16        - name: Checkout repository
17          uses: actions/checkout@v4
18
19        - name: Set up JDK 25
20          uses: actions/setup-java@v4
21          with:
22            distribution: temurin
23            java-version: '25'
24            cache: maven
25
26        - name: Verify Maven installation
27          run: mvn -version
28
29        - name: Install dependencies and build
30          run: mvn -B clean compile
31
32        - name: Run Checkstyle
33          run: mvn -B checkstyle:check
34
35        - name: Run unit tests
36          run: mvn -B test
37
```

Цей файл відповідає за автоматичний запуск при push у гілку main, запуск при створенні Pull Request, встановлення JDK, компіляція проєкту, перевірка стилю коду (Checkstyle), виконання всіх unit-тестів. Якщо тест не проходить або є помилки стилю, то pipeline завершується зі статусом failed.

Етап 3 (CD):

Було реалізовано автоматичне створення релізів за допомогою окремого workflow у директорії віддаленого репозиторію `.github/workflows/release.yml`.

```
1  name: Release
2
3  on:
4    push:
5      tags:
6        - 'v*.*.*'
7
8  permissions:
9    contents: write
10
11 jobs:
12   release:
13     runs-on: ubuntu-latest
14
15     steps:
16       - name: Checkout repository
17         uses: actions/checkout@v4
18
19       - name: Set up JDK 25
20         uses: actions/setup-java@v4
21         with:
22           distribution: temurin
23           java-version: '25'
24           cache: maven
25
26       - name: Run Checkstyle
27         run: mvn -B checkstyle:check
28
29       - name: Run unit tests
30         run: mvn -B test
31
32       - name: Build JAR
33         run: mvn -B clean package -DskipTests
34
35       - name: Upload workflow artifact
36         uses: actions/upload-artifact@v4
37         with:
38           name: package-manager-jar
39           path: target/PackageManager-1.0-SNAPSHOT.jar
40
41       - name: Create GitHub Release and upload JAR
42         env:
43           GH_TOKEN: ${ secrets.GITHUB_TOKEN }
44         run: |
45           gh release create "${GITHUB_REF_NAME}" \
46             target/PackageManager-1.0-SNAPSHOT.jar \
47             --generate-notes
48
```

Workflow запускається при створенні тегу версії `git tag v1.0.0`. Цей файл відповідає за перевірку стилю, запуск всіх unit-тестів, збірка фінального `.jar` файлу, створення GitHub Release, до релізу прикріплюється `.jar` як артефакт. Це дозволяє користувачам завантажити готову програму та запускати її без необхідності збірки.

Висновок:

У результаті виконання роботи було:

1. реалізовано контейнеризацію застосунку за допомогою Docker;
2. налаштовано CI для автоматичної перевірки якості коду;
3. реалізовано CD для автоматичного створення релізів;
4. забезпечено можливість зручного розповсюдження програми у вигляді `.jar`.